

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V) Artificial
Intelligence Application Development

Module 1

Practical –VII Part 6,7,8

Roll no. : T023	Name : Som Mishra
Class : TYIT	Batch : 1
Date of assignment : 08/08/2026	Date of assignment : 08/08/2026

6. Implement Decision Tree regression on a regression dataset.

Code-:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load dataset
data = pd.read_csv("iris (2)(1).csv")

# Input and output
X = data[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
y = data['species']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

# Create model
model = DecisionTreeClassifier(random_state=1)
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Prediction
new_data = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=['sepal_length', 'sepal_width',
            'petal_length', 'petal_width']
)
```

```
prediction = model.predict(new_data)
print("Predicted Species:", prediction[0] )
```

Output:-

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
```

```
Predicted Species: setosa
```

7. Implement k-Nearest Neighbors (kNN) classifier.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Prediction
prediction = model.predict([X_test[0]])

print("Predicted Class:", prediction[0])
```

Output-:

```
Training Accuracy: 95.23809523809523
Testing Accuracy: 97.77777777777777
Predicted Class: 0
```

8. Implement multiclass classification using Logistic Regression or Decision Tree

Code-:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=1
)

# Create Decision Tree model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Predict one sample
prediction = model.predict([X_test[0]])

print("Predicted Class:", prediction[0])
```

Output-:

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
```

Task -:

Q1. Perform Logistic Regression, Decision Tree, and kNN on the same dataset. Compare the Results.

Code-:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=1
)

# 1. Logistic Regression

logistic_model = LogisticRegression(max_iter=200)

logistic_model.fit(X_train, y_train)

logistic_train = logistic_model.score(X_train, y_train) * 100
logistic_test = logistic_model.score(X_test, y_test) * 100

# 2. Decision Tree
tree_model = DecisionTreeClassifier(random_state=1)

tree_model.fit(X_train, y_train)

tree_train = tree_model.score(X_train, y_train) * 100
tree_test = tree_model.score(X_test, y_test) * 100
```

```
--  
# 3. kNN  
  
knn_model = KNeighborsClassifier(n_neighbors=3)  
  
knn_model.fit(X_train, y_train)  
  
knn_train = knn_model.score(X_train, y_train) * 100  
knn_test = knn_model.score(X_test, y_test) * 100  
  
# Display Results  
print("LOGISTIC REGRESSION")  
print("Training Accuracy:", logistic_train)  
print("Testing Accuracy:", logistic_test)  
  
print("\nDECISION TREE")  
print("Training Accuracy:", tree_train)  
print("Testing Accuracy:", tree_test)  
  
print("\nkNN")  
print("Training Accuracy:", knn_train)  
print("Testing Accuracy:", knn_test)
```

Output:-

```
LOGISTIC REGRESSION  
Training Accuracy: 100.0  
Testing Accuracy: 88.88888888888889  
  
DECISION TREE  
Training Accuracy: 100.0  
Testing Accuracy: 88.88888888888889  
  
kNN  
Training Accuracy: 100.0  
Testing Accuracy: 100.0
```